

FedEx AI Logistics Command Center

Conversation notes and implementation guide for connecting Python analysis, LLM insights, and Power BI dashboard visuals.

Project Type	AI-powered logistics dashboard
Main Tools	Python, LLM, Power BI
Primary Goal	Make AI-generated insights appear directly on the dashboard
Prepared For	Shruti Auti
Date	16 Jul 2026

1. Core Idea

The existing FedEx dashboard should not be treated as only a normal reporting page. The stronger project idea is to convert it into a FedEx AI Logistics Command Center, where Python performs data analysis, an LLM explains what the numbers mean, and Power BI displays the final insights directly on the dashboard.

Short version: Python is the analyst, the LLM is the business consultant, and Power BI is the visual interface.

Recommended Architecture

```
FedEx Dataset
-> Python Analysis
-> KPI Summary
-> LLM Insights
-> Final DataFrame or CSV
-> Power BI Dashboard
```

2. Why Python Analysis Is Needed

The LLM should not receive the full dataset. Python should first calculate important logistics KPIs, then only the summarized results should be passed to the LLM. This keeps the solution faster, cheaper, cleaner, and more reliable.

- Python calculates total revenue, profit, delayed shipments, on-time percentage, average delivery days, region-level issues, and health score.
- The LLM converts those KPI values into readable executive insights, risks, opportunities, and recommendations.
- Power BI displays the final output as cards, text panels, recommendations, alerts, and command-center visuals.

3. Where the Result Appears

The result should appear directly on the Power BI dashboard. The user should not need to open the Python terminal, CSV file, or raw DataFrame. Power BI should consume the Python output and show it as normal dashboard visuals.

```
Power BI Fields Pane
```

```
result
- HealthScore
- Revenue
- Profit
- OnTimePercentage
- AIInsight
- Risk
- Recommendation
```

These fields can be placed into Power BI visuals such as Card, Multi-row Card, Table, Smart Narrative, Text Box, or custom styled sections.

4. Dashboard Mockup

Below is the visual structure discussed for the AI-powered FedEx dashboard.

FEDEX AI LOGISTICS COMMAND CENTER			
Revenue	Profit	On-Time %	Health
\$1.25M	\$210K	87%	84 / 100
AI INSIGHTS			
Warning: Delivery delays increased in West Region			
Warning: Warehouse utilization approaching capacity			
Positive: Revenue continues growing despite delays			
AI RECOMMENDATIONS			
1. Increase fleet allocation in West Region			
2. Optimize last-mile delivery routes			
3. Review warehouse processing bottlenecks			

Suggested Visual Mapping

Python / LLM Output	Power BI Visual
HealthScore	Card visual
Revenue, Profit, OnTimePercentage	KPI cards or multi-row cards
AllInsight	Smart Narrative, text box, or table visual
Risk	Alert panel or table visual
Recommendation	Recommendation tile or multi-row card

5. Direct Python Script in Power BI

If you use Power BI Desktop > Get Data > Python Script, Power BI runs the script and detects output DataFrames. If your script ends with a DataFrame named result, Power BI shows result in the Navigator. After loading it, fields appear in the Fields Pane and can be used in visuals.

```
import pandas as pd

# Load FedEx data
fedex = pd.read_csv(r"C:\Data\fedex.csv")

# Basic analysis
revenue = fedex["Revenue"].sum()
profit = fedex["Profit"].sum()
delayed_shipments = len(fedex[fedex["Delivery Status"] == "Delayed"])
total_shipments = len(fedex)
on_time_percentage = ((total_shipments - delayed_shipments) / total_shipments) * 100

# Simple health score
health_score = 100
if on_time_percentage < 95:
```

```
        health_score -= 10
    if on_time_percentage < 90:
        health_score -= 10
    if profit < 0:
        health_score -= 20

# DataFrame that Power BI will load
result = pd.DataFrame({
    "HealthScore": [health_score],
    "Revenue": [revenue],
    "Profit": [profit],
    "DelayedShipments": [delayed_shipments],
    "OnTimePercentage": [round(on_time_percentage, 2)]
})

result
```

6. Python Analysis + LLM Call

For the AI version, Python calculates the KPIs first, then sends only the KPI summary to the LLM. The LLM returns business-friendly insight text. Power BI can display that text on the dashboard.

```
import pandas as pd
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

fedex = pd.read_csv(r"C:\Data\fedex.csv")

revenue = fedex["Revenue"].sum()
profit = fedex["Profit"].sum()
delayed_shipments = len(fedex[fedex["Delivery Status"] == "Delayed"])
total_shipments = len(fedex)
on_time_percentage = ((total_shipments - delayed_shipments) / total_shipments) * 100

health_score = 100
if on_time_percentage < 95:
    health_score -= 10
if on_time_percentage < 90:
    health_score -= 10
if profit < 0:
    health_score -= 20

kpi_summary = f"""
Revenue: {revenue}
Profit: {profit}
Delayed Shipments: {delayed_shipments}
On-Time Percentage: {on_time_percentage:.2f}
Health Score: {health_score}
"""

prompt = f"""
You are a logistics AI consultant.
Analyze these FedEx dashboard KPIs:

{kpi_summary}

Generate:
1. Executive summary
2. Top risks
3. Opportunities
4. Recommendations

Keep it concise and business friendly.
"""

response = client.chat.completions.create(
    model="gpt-4.1-mini",
    messages=[
        {"role": "system", "content": "You are a logistics analytics expert."},
        {"role": "user", "content": prompt}
    ]
)

ai_insight = response.choices[0].message.content

result = pd.DataFrame({
    "HealthScore": [health_score],
    "Revenue": [revenue],
    "Profit": [profit],
    "OnTimePercentage": [round(on_time_percentage, 2)],
    "AIInsight": [ai_insight]
})
```

result

Important Note

If this script is run inside Power BI, the LLM API call can execute again whenever the dataset refreshes. For a simpler and more stable first version, you can run the Python script outside Power BI and save `ai_insights.csv`, then connect Power BI to that CSV.

7. Cleaner MVP Recommendation

For the first working version, use this flow:

```
generate_ai_insights.py
-> reads FedEx data
-> calculates KPIs
-> calls LLM
-> saves ai_insights.csv
-> Power BI reads ai_insights.csv
```

This approach is easier to debug, avoids repeated API calls from Power BI, and lets the dashboard refresh quickly.

8. Final Project Description

A strong portfolio description would be: I developed an AI-powered FedEx Logistics Command Center where Python performs operational analytics, an LLM generates executive insights and recommendations, and Power BI visualizes the results through a command-center dashboard.